SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		SLOT B
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	P Mohan Babu	**Reg. No.:**	192472254
Section / Batch:	CSE AI	**Date:**	17-08-2026

Code of Conduct

I P Mohan Babu / 192472254 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P Mohan Babu

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.
Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.
Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.
Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.
Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT TOOL 3 – COMPARATIVE ANALYSIS
Course Code: DSA03
Course Title: Natural Language Processing
CO Assessed: CO4 – Precision
Formulate context-free grammars, feature structures, probabilistic parsing techniques and construct parsing frameworks.

Q1. CFG Trees vs Dependency Parsing
A sentence can be represented using either a **Context-Free Grammar (CFG) tree** or a **dependency parsing structure**. Both describe syntactic structure, but they focus on different aspects.
CFG Representation
Consider:
"The student reads the book."
A simplified CFG is:

```
S
├── NP
│   ├── Det → The
│   └── N → student
└── VP
    ├── V → reads
    └── NP
        ├── Det → the
        └── N → book
```

The CFG representation emphasizes **constituents or phrases**. It shows that:

- "The student" forms a noun phrase.
- "the book" forms another noun phrase.
- The verb phrase contains the verb and its object.

CFG is useful for grammar checking, phrase structure analysis and syntactic generation.
Dependency Representation
A dependency parser focuses on relationships between individual words:

```
          reads
         /     \
    student     book
      |          |
     The        the
```

The main relations are:
reads → student = subject
reads → book = object
student → The = determiner
book → the = determiner
The verb **"reads"** acts as the central head, while the other words depend on it.
Comparison

Aspect	**CFG Tree**	**Dependency Parsing**
Basic unit	Phrase/constituent	Word
Main focus	Phrase structure	Word-to-word relationships
Representation	Hierarchical tree	Dependency graph/tree
Subject/object relations	Indirectly represented	Directly represented
Phrase structure	Strong	Less explicit
Long-distance relationships	Can be complex	Often represented directly
Grammar generation	Very useful	Less focused on generation
Semantic relation extraction	Requires additional processing	Often easier
Information extraction	Moderate	Strong
Grammar checking	Strong	Strong for grammatical relations

Which Is Better for Capturing Word Relationships?

Dependency parsing is generally more effective for directly capturing relationships between words.
For example:
"The doctor prescribed medicine to the patient."
Dependency parsing can directly represent:
prescribed
├── doctor → subject/agent
├── medicine → object/theme
└── patient → indirect object/recipient
This makes dependency structures particularly useful for:

- Information extraction
- Semantic role analysis
- Relation extraction
- Question answering
- Machine translation
- Search systems

However, CFG remains valuable when the application specifically requires **constituent structure**, such as grammar checking, syntactic generation and phrase-level analysis.

Conclusion

For **capturing relationships between words**, dependency parsing is preferable because grammatical dependencies are explicitly represented. CFG is more suitable when the system needs detailed phrase and constituent structure. In practical NLP systems, the two approaches can also complement each other.

Q2. Top-Down Parsing vs Earley Parsing

A real-time application needs to process incomplete and ambiguous user input. The two approaches being compared are **top-down parsing** and **Earley parsing**.

Top-Down Parsing

Top-down parsing starts from the grammar's start symbol and attempts to derive the input.
For example:
S
↓
VP
↓
V NP
↓
Book NP
It predicts the structure before fully observing the input.

Advantages

- Conceptually simple.
- Easy to implement for small grammars.
- Works efficiently with well-designed predictive grammars.
- Suitable for controlled language.

Limitations

Top-down parsing can suffer from:

- Backtracking.
- Repeated computation.
- Problems with left-recursive grammars.
- Difficulty handling highly ambiguous structures.
- Reduced efficiency for large grammars.
- Difficulty handling incomplete input.

Earley Parsing

Earley parsing is a chart-parsing algorithm designed to handle general CFGs.
It uses three main operations:

1. **Prediction**
2. **Scanning**
3. **Completion**

For an incomplete input such as:

"Book a flight to..."

the parser can maintain partial states instead of requiring the complete sentence before processing.

A simplified state might be:

[VP → V NP •, 0]

The chart stores previously computed states so that they do not need to be recomputed repeatedly.

Comparison

Aspect	Top-Down Parsing	Earley Parsing
Starting point	Start symbol	Start symbol with chart states
Ambiguity	Backtracking may increase	Maintains multiple hypotheses
Incomplete input	Less suitable	Well suited
Left recursion	Problematic in simple implementations	Supported
Repeated work	More likely	Reduced through chart
General CFG	Depends on implementation	Supported
Memory	Generally lower	Higher
Real-time dynamic input	Less suitable	More suitable

For a general CFG, Earley parsing has worst-case complexity:

[
O(n^3)
]

while it can be much more efficient for practical grammars.

Example

Suppose a voice assistant receives:

"Book a flight to..."

Top-down parsing may predict many possible noun phrases and productions, resulting in backtracking when the input changes.

Earley parsing can retain partial structures and continue parsing when additional words arrive:

"Delhi"

followed by:

"with a window seat."

This makes it more suitable for dynamic speech and conversational systems.

Conclusion

Earley parsing is more suitable for incomplete and ambiguous real-time input. Although it requires more memory, its chart-based processing reduces unnecessary recomputation and supports general CFGs and partial parses. Top-down parsing remains useful for small, predictable grammars where simplicity and low resource consumption are more important.

Q3. CFG vs PCFG vs Neural Parsing

Consider:

“She saw the man with a telescope.”

The sentence has **prepositional phrase attachment ambiguity**.

Interpretation 1 – Telescope Used by "She"

She
└── saw
    └── man
        └── with a telescope

Semantic interpretation:

She used a telescope to see the man.

Interpretation 2 – Man Has a Telescope

She
└── saw
    └── man
        └── with a telescope

The PP can instead be interpreted as modifying **the man**.

Semantic interpretation:

She saw a man who had a telescope.
The surface words alone do not completely determine the intended meaning.

---

1. CFG
A CFG can generate both valid structures:
VP → VP PP
NP → NP PP
Thus:
VP → saw NP PP
and:
NP → man PP
can both be possible.
Strength
CFG provides the possible syntactic structures.
Limitation
A basic CFG does not inherently know which interpretation is more likely.
Therefore, it can identify ambiguity but cannot reliably resolve it by itself.

---

2. PCFG
A Probabilistic CFG associates probabilities with grammar rules.
For example:
VP → VP PP [0.65]
NP → NP PP [0.35]
If the VP attachment has a higher probability in the training domain, the parser can prefer it.
The probability of a parse tree can be approximated by:
[
P(T)=\prod_i P(R_i)
]
where (R_i) represents each grammar production used by the parse.
Strengths

- Resolves ambiguity using probabilities.
- More realistic than equally weighted CFG alternatives.
- Relatively interpretable.
- Useful when sufficient treebank data are available.

Limitations

- Performance depends on the quality of the training data.
- Basic PCFGs may not capture rich contextual information.
- Rule probabilities may not be enough for highly context-dependent decisions.

---

3. Neural Parsing
Neural parsers use learned representations from models such as transformer-based architectures.
They can use context from the entire sentence to determine the likely interpretation.
For example, additional context:
“She saw the man with a telescope while observing the stars.”
strongly favors the interpretation that **she used the telescope**.
A neural parser can consider:

- Surrounding words.
- Sentence-level context.
- Semantic information.
- Large-scale training data.
- Lexical and contextual representations.

Comparison

Aspect	**CFG**	**PCFG**	**Neural Parsing**
Represents syntax	Strong	Strong	Strong
Handles ambiguity	Generates alternatives	Uses probabilities	Uses learned contextual

Aspect	CFG	PCFG	Neural Parsing
			representations
Context sensitivity	Low	Moderate	High
Training requirement	Grammar rules	Treebank/probabilities	Large training data
Interpretability	High	High/Moderate	Lower
Domain adaptation	Manual grammar changes	Retraining probabilities	Fine-tuning/adaptation
Complex language	Limited	Better	Strong
Computational requirements	Low	Moderate	High

Most Effective Approach

For **real-world applications, neural parsing is generally the most powerful**, particularly when large amounts of training data and computational resources are available.

However, a hybrid architecture can be even more practical:

Input
↓
Neural Contextual Representation
↓
Candidate Syntactic Structures
↓
Probabilistic Ranking
↓
Semantic Interpretation

CFG can still provide explicit grammatical constraints, while PCFG-style probabilities can improve interpretability and ranking.

Conclusion

CFG is useful for generating possible structures, PCFG improves ambiguity resolution through probability, and neural parsing provides stronger contextual understanding. For modern large-scale NLP applications, **neural parsing or a hybrid neural-probabilistic parser** is generally the preferred solution.

Q4. Feature Structures vs Subcategorization Frames

A grammar system must enforce:

1. Subject–verb agreement.
2. Verb argument structures.

The two techniques being compared are **feature structures** and **subcategorization frames**.

Feature Structures

Feature structures represent grammatical properties using attribute-value pairs.

For example:

Subject:
  NUMBER = singular
  PERSON = third
Verb:
Verb:
  NUMBER = singular
  PERSON = third

The sentence:

“The student reads the book.”

is valid because:

Subject.NUMBER = Verb.NUMBER
Subject.PERSON = Verb.PERSON

But:

“The student readss the book.”

would violate the lexical/grammatical constraints.

Feature structures can represent:

- Number
- Person
- Gender
- Tense
- Case
- Agreement
- Definiteness
- Other linguistic properties

Subcategorization Frames
Subcategorization specifies which arguments a verb requires.
For example:
"eat"
eat:
Subject + Object
Example:
“The patient takes medicine.”
take
├── Subject = patient
└── Object = medicine
"give"
give:
Subject + Object + Recipient
Example:
“The doctor gave medicine to the patient.”
give
├── Subject = doctor
├── Object = medicine
└── Recipient = patient
Subcategorization therefore ensures that the verb receives the expected arguments.

Comparison

Aspect	**Feature Structures**	**Subcategorization Frames**
Agreement	Excellent	Not primary purpose
Number/person	Strong	Limited
Verb arguments	Can encode	Excellent
Argument count	Possible	Explicit
Verb valency	Moderate	Excellent
Tense/features	Strong	Limited
Semantic roles	Can be integrated	Can be associated with arguments
Grammatical constraints	Strong	Focused on verb requirements

Which Provides Better Grammatical Support?
For **subject–verb agreement**, feature structures are clearly superior.
For **verb argument structures**, subcategorization frames are superior.
Therefore, choosing only one would not be ideal.
The best architecture is:
Feature Structures
+
Subcategorization Frames
↓
Constraint-Based Grammar
For example:
“The doctors prescribe medicines.”
Feature structures check:
doctors → NUMBER = plural

prescribe → NUMBER = plural
Subcategorization checks:
prescribe:
  Subject + Object
Thus the complete sentence satisfies both agreement and argument requirements.

Conclusion

Feature structures are better for grammatical agreement constraints, while subcategorization frames are better for verb argument structures. A combined approach provides the strongest grammatical representation and is preferable for accurate NLP systems.

Q5. Transition-Based vs Graph-Based Dependency Parsing

Large-scale NLP systems must parse massive datasets quickly while maintaining acceptable accuracy.
The two approaches are:

- **Transition-based parsing**
- **Graph-based parsing**

---

1. Transition-Based Parsing

Transition-based parsing builds a dependency tree incrementally using a sequence of decisions.
A common parser uses:

- Stack
- Buffer
- Set of dependency relations

For example:
Stack Buffer
[ROOT] [The, doctor, prescribed, medicine]
The parser applies transitions such as:

- SHIFT
- LEFT-ARC
- RIGHT-ARC

Example:
SHIFT
→ [ROOT, The]

SHIFT
→ [ROOT, The, doctor]

...
The parser gradually constructs the dependency tree.

Advantages

- Fast inference.
- Often approximately linear in sentence length.
- Low memory requirements.
- Suitable for real-time applications.
- Easy to scale across large datasets.

Limitations

- Decisions are often made locally.
- An early incorrect decision can affect later parsing.
- Greedy approaches may suffer from error propagation.
- Accuracy may decrease on highly complex structures.

---

2. Graph-Based Parsing

Graph-based parsing treats dependency parsing as an optimization problem.
Each possible dependency arc receives a score:
[
Score(h,m)
]

where:

- (h) = head
- (m) = dependent

The parser searches for the dependency tree with the highest total score:

[
T^*=\arg\max_T Score(T)
]

For example:

```
      prescribed
     /          \
  doctor        medicine
```

The system compares possible dependency structures and selects the highest-scoring valid tree.

Advantages

- Considers global tree structure.
- Often more robust against local errors.
- Better for complex dependency relationships.
- Can produce high parsing accuracy.

Limitations

- More computationally expensive.
- Requires more memory.
- Global optimization can increase latency.
- Scaling to very large datasets can be more challenging.

Comparison

Aspect	Transition-Based	Graph-Based
Parsing strategy	Sequential decisions	Global optimization
Decision-making	Local/incremental	Global
Speed	Very high	Lower
Memory	Lower	Higher
Error propagation	Higher in greedy systems	Generally lower
Long-distance relations	Can be challenging	Better global modeling
Accuracy	High with good models	Often strong
Real-time use	Excellent	Less suitable
Massive datasets	Excellent scalability	More computationally expensive
Complexity	Usually near-linear in practice	Often higher

Example

For:

“The doctor prescribed medicine to the patient.”

A transition-based parser might make a sequence of decisions:

SHIFT → The
SHIFT → doctor
SHIFT → prescribed
RIGHT-ARC → prescribed → doctor
SHIFT → medicine
RIGHT-ARC → prescribed → medicine
...

This is fast because the parser makes one decision at a time.

A graph-based parser instead evaluates candidate dependencies and attempts to find the globally best structure:

```
prescribed
├── doctor
├── medicine
└── patient
```

The graph-based approach may produce a better global decision but requires more computation.

Which Is Better for Large-Scale Applications?

For **massive datasets and high-throughput processing**, transition-based parsing is generally more suitable because:

- It has low per-sentence computational overhead.
- It can process sentences incrementally.
- It requires relatively little memory.
- It is well suited to parallel and batch processing.
- It provides low latency.

Graph-based parsing is preferable when **maximum structural accuracy is more important than throughput**, especially for complex linguistic structures.

Recommended Industry Architecture

A modern system can combine the advantages of both:

Large Dataset
↓
Neural Encoder
↓
Transition-Based Parser
↓
Fast Dependency Trees
↓
Confidence / Quality Filter
↓
Graph-Based Reanalysis
for difficult cases

This creates a **fast-first, accuracy-focused fallback architecture**.

Conclusion

Transition-based parsing is generally the better choice for large-scale, high-throughput NLP systems, while graph-based parsing is valuable when complex structures require more global analysis. A hybrid system can use transition-based parsing for most sentences and graph-based reanalysis for difficult or low-confidence cases.

Overall Comparative Conclusion

The five comparisons demonstrate that NLP systems often benefit from combining complementary techniques rather than selecting one method exclusively.

Problem	Preferred Approach	Reason
Word relationships	Dependency parsing	Direct dependency relations
Incomplete/ambiguous input	Earley parsing	Chart-based partial and ambiguous parsing
Real-world ambiguity	Neural/hybrid parsing	Strong contextual representation
Agreement + argument structure	Feature structures + subcategorization	Complementary grammatical constraints
Massive-scale parsing	Transition-based parsing	High speed and scalability

The key engineering principle is to select a parsing technique according to the **application's requirements for accuracy, ambiguity handling, latency, memory, interpretability and scalability**. No single parsing method is optimal for every NLP task. Hybrid architectures are often the most practical solution for modern industry systems.